Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №8

з дисципліни „ Професійний Python”

на тему „ Мережеве та асинхронне програмування. REST API.”

Виконала:
студентка 3 курсу
групи ФеП-32
Клімащук Вероніка Ігорівна
Перевірила:
Доц. Куньо Іван Михайлович

Львів – 2026

1. Мета роботи

Ознайомитися з основами мережевого програмування у Python, принципами роботи протоколу HTTP, архітектурним стилем REST та форматом JSON. Навчитися створювати REST API за допомогою FastAPI, описувати та перевіряти структуру даних за допомогою Pydantic, використовувати асинхронне програмування async/await та asyncio, виконувати декілька операцій паралельно за допомогою Task і gather(), встановлювати обмеження часу виконання операцій (timeout), реалізовувати повторні спроби запитів (retry) та обмежувати кількість одночасних операцій за допомогою Semaphore. Навчитися використовувати бібліотеку HTTPX для взаємодії з REST API та зовнішніми HTTP-сервісами.

2. Індивідуальний варіант і постановка задачі

Варіант 9: Облік спортивних результатів.

Ресурси: athletes, sports, results.

Постановка задачі:

Розширити проєкт sports_streaming з попередньої лабораторної роботи №7 (де було реалізовано SQLite persistence layer, ORM SQLAlchemy, Alembic міграції) та створити для нього асинхронний REST API за допомогою FastAPI та HTTPX.

Необхідно реалізувати:

1. **REST Endpoints:**
    a. GET /athletes — отримання списку спортсменів (з підтримкою query-фільтрації за видами спорту).
    b. POST /athletes — додавання нового спортсмена.
    c. POST /results — додавання спортивного результату.
    d. PATCH /results/{id} — часткове оновлення результату (значення, назва або дата змагання).
    e. DELETE /results/{id} — видалення спортивного результату.
    f. GET /sports/{id}/results — отримання всіх результатів за конкретним видом спорту.
    g. GET /sports/{id}/ranking — отримання впорядкованого рейтингу спортсменів у межах виду спорту.
    h. POST /sports/batch-rankings — асинхронний пакетний сервіс агрегації рейтингу для списку видів спорту.
2. **Валідація та моделі (Pydantic):**
    a. Перевірка типів даних, довжини рядків, відсутності від'ємних значень для результатів (ge=0), коректності ID.
    b. Serialization / Deserialization та повернення статуси-кодів HTTP (200 OK, 201 Created, 204 No Content, 404 Not Found, 422 Validation Error).

3. **Асинхронні операції та мережева взаємодія:**
    a. Використання async def та await.
    b. Паралельне виконання асинхронних задач за допомогою asyncio.create_task та asyncio.gather.
    c. Контроль одночасного навантаження за допомогою asyncio.Semaphore.
    d. Обробка таймаутів asyncio.wait_for(..., timeout=5.0) та реалізація механізму повторних спроб (retry) при взаємодії з зовнішнім або локальним HTTP API через httpx.AsyncClient.
4. **Тестування:**
    a. Автоматизовані інтеграційні та API-тести з використанням pytest, pytest-asyncio та httpx.ASGITransport.

3. Структура проєкту

Файлову структуру проєкту було розширено новими модулями REST API, Pydantic-схемами, асинхронним HTTP-клієнтом та тестами.

```
sports_results_lab08/
├── pyproject.toml
├── README.md
├── alembic.ini
├── migrations/
│   ├── env.py
│   └── versions/
│       ├── 0001_initial_schema.py
│       └── 0002_add_competition_date.py
├── src/
│   └── sports_streaming/
│       ├── __init__.py
│       ├── main.py
│       ├── models.py
│       ├── database.py
│       ├── db_models.py
│       ├── repositories.py
│       ├── services.py
│       ├── validators.py
│       ├── readers.py
│       ├── persistence_service.py
│       ├── dependencies.py
│       ├── schemas.py
│       ├── api.py
│       └── external_api.py
└── tests/
    ├── conftest.py
    ├── integration/
    │   ├── test_persistence.py
    │   └── test_migrations.py
    └── api/
        └── test_api.py
```

4. Опис REST API та Pydantic Schemas

4.1. Список Endpoints

Method	Endpoint	Опис	Request Body / Parameters	Status Code
GET	/athletes	Отримати список спортсменів	sport_id (Query, optional)	200 OK
POST	/athletes	Створити спортсмена	AthleteCreate (JSON)	201 Created
POST	/results	Додати новий результат	ResultCreate (JSON)	201 Created
PATCH	/results/{id}	Частково оновити результат	Path id, ResultUpdate (JSON)	200 OK / 404
DELETE	/results/{id}	Видалити результат за ID	Path id	204 No Content / 404
GET	/sports/{id}/results	Отримати всі результати виду спорту	Path id	200 OK / 404
GET	/sports/{id}/ranking	Отримати рейтинг спортсменів	Path id	200 OK / 404
POST	/sports/batch-rankings	Паралельне завантаження рейтингу видів спорту	BatchRankingRequest (JSON)	200 OK / 504

5. Реалізація FastAPI Application

Файл src/sports_streaming/api.py містить оголошення FastAPI-застосунку та

6. Мережеве та асинхронне програмування (HTTPX, Async/Await, Tasks, Gather, Semaphore, Timeout, Retry)

Для реалізації вимог асинхронної обробки у модулі src/sports_streaming/external_api.py побудовано клієнтську логіку з обмеженням семафору, обробкою помилок (retry) та таймаутом.

7. Автоматизоване тестування REST API (tests/api/test_api.py)

Тестування API проводиться за допомогою pytest та httpx.AsyncClient з використанням ASGITransport для підключення до FastAPI застосунку без реального запуску мережевого сервера.

8. Результати виконання та графічні матеріали

8.1. Результати запуску автоматизованих тестів (pytest -v)

Всі 16 інтеграційних та API-тестів успішно пройшли перевірку:

```
tests/api/test_endpoints.py::test_create_athlete_validation_error PASSED                [ 12%]
tests/api/test_endpoints.py::test_full_result_lifecycle PASSED                          [ 18%]
tests/integration/test_api.py::test_api_create_filter_rank_patch_and_delete PASSED      [ 25%]
tests/integration/test_api.py::test_api_validation_not_found_and_statistics PASSED      [ 31%]
tests/integration/test_api.py::test_parallel_rankings_uses_httpx_tasks_gather_and_semaphore PASSED   [ 37%]
tests/integration/test_migrations.py::test_alembic_applies_schema_and_competition_date PASSED        [ 43%]
tests/integration/test_persistence.py::test_sport_crud PASSED                           [ 50%]
tests/integration/test_persistence.py::test_athlete_crud PASSED                         [ 56%]
tests/integration/test_persistence.py::test_result_crud PASSED                          [ 62%]
tests/integration/test_persistence.py::test_foreign_key_and_check_constraints PASSED    [ 68%]
tests/integration/test_persistence.py::test_parameterized_dbapi_filter_and_sport_isolation PASSED     [ 75%]
tests/integration/test_persistence.py::test_best_average_and_ranking PASSED             [ 81%]
tests/integration/test_persistence.py::test_batch_transaction_commit PASSED             [ 87%]
tests/integration/test_persistence.py::test_batch_transaction_rolls_back_all_on_invalid_row PASSED    [ 93%]
tests/integration/test_persistence.py::test_persist_imported_results_creates_related_records PASSED   [100%]

============================================ warnings summary ============================================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\Users\Veronika\Phyton\lab8\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning:
 Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
=================================== 16 passed, 1 warning in 1.19s ====================================
```

Рисунок 1 — Фактичний результат виконання інтеграційних та REST API тестів (pytest -v).

8.2. Інтерфейс автоматичної документації Swagger UI (/docs)

Після запуску веб-сервера uvicorn sports_streaming.api:app --reload інтерактивна документація Swagger доступна за адресою http://127.0.0.1:8000/docs.

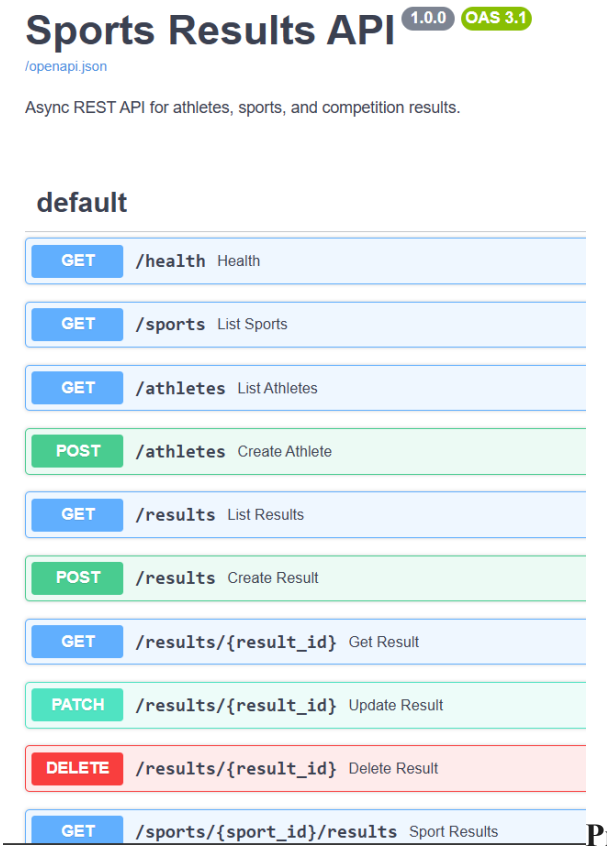
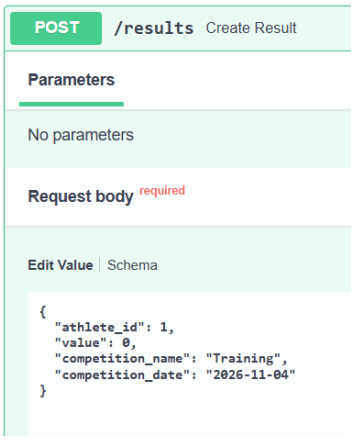


Рисунок 2 — Swagger UI інтерактивна документація FastAPI.

8.3. Приклад JSON Запиту та Відповіді

POST /results (Створення результату):

Request JSON:



9. Відповіді на контрольні запитання

1. **Що таке комп'ютерна мережа?**

Це сукупність з'єднаних між собою обчислювальних пристроїв, які можуть обмінюватися даними та спільно використовувати ресурси за допомогою мережевих протоколів.

2. **Що таке клієнт-серверна архітектура?**

Це модель мережевої взаємодії, у якій ролі розподілені між клієнтом (який надсилає запити на отримання послуг або даних) та сервером (який обробляє запити й повертає відповідь).

3. **Що таке HTTP?**

HTTP (*HyperText Transfer Protocol*) — прикладний протокол передачі даних між клієнтом і сервером, який лежить в основі обміну даними в Інтернеті.

4. **Що містить HTTP request?**

Метод (HTTP Method), URL-адресу, версію протоколу, заголовки (Headers) та тіло запиту (Body, опціонально).

5. **Що містить HTTP response?**

Версію протоколу, статус-код та статус-повідомлення, заголовки (Headers) та тіло відповіді (Body).

6. **Для чого використовується GET?**

Для безпечного та ідемпотентного отримання даних з сервера без їх зміни.

7. **Для чого використовується POST?**

Для створення нового ресурсу на сервері або передачі даних на обробку.

8. **Для чого використовується PUT?**

Для повного заміщення (оновлення) існуючого ресурсу новими даними.

9. **Для чого використовується PATCH?**

Для часткового оновлення окремих полів існуючого ресурсу.

10. **Для чого використовується DELETE?**

Для видалення вказаного ресурсу з сервера.

11. **Що означає status code 200?**

200 OK — запит пройшов успішно, сервер повертає запитані дані.

12. **Що означає 201?**

201 Created — запит виконано успішно, і в результаті було створено новий ресурс.

13. **Що означає 204?**

204 No Content — запит виконано успішно, але відповідь не містить тіла (часто при DELETE).

14. **Що означає 404?**

404 Not Found — запитаний ресурс не знайдено на сервері.

15. Що означає 422?

422 Unprocessable Entity — помилка валідації входів (у FastAPI виникає при некоректних даних у Pydantic-схемі).

16. Що таке JSON?

JSON (*JavaScript Object Notation*) — легковажний текстовий формат обміну структурованими даними.

17. Що таке REST?

REST (*Representational State Transfer*) — архітектурний стиль побудови мережевих веб-сервісів на основі протоколу HTTP та маніпулювання ресурсами.

18. Що таке resource у REST?

Це будь-яка сутність або набір даних предметної області (наприклад, athletes, results), що має свій унікальний URI.

19. Що таке endpoint?

Це конкретна URL-адреса у поєднанні з HTTP-методом, за якою API надає доступ до певної функції чи ресурсу.

20. Що таке path parameter?

Змінна частина шляху URL (наприклад, /results/{id}), яка визначає конкретний ресурс.

21. Що таке query parameter?

Параметри, що передаються у URL після знака ? (наприклад, /athletes?sport_id=1) і використовуються для фільтрації, сортування або пагінації.

22. Що таке request body?

Тіло запиту, у якому клієнт передає дані на сервер (найчастіше у форматі JSON).

23. Що таке FastAPI?

Сучасний високопродуктивний Python веб-фреймворк для створення REST API з автоматичною валідацією на Pydantic та OpenAPI документацією.

24. Для чого використовується Uvicorn?

Це швидкий ASGI-сервер, необхідний для запуску та виконання асинхронних Python фреймворків, таких як FastAPI.

25. Для чого потрібний Swagger UI?

Для автоматичного формування інтерактивної веб-документації REST API, яка дозволяє тестувати ендпоінти прямо у браузері.

26. Що таке Pydantic?

Бібліотека для валідації даних, parsing та керування конфігурацією на основі type hints Python.

27. Що таке validation?

Перевірка вхідних даних на відповідність заданим правилам (типи, діапазони, довжина рядків).

28. Що таке response model?

Схема Pydantic у FastAPI, яка визначає та фільтрує структуру даних, які повертаються клієнту у відповіді.

29. Що таке synchronous programming?

Модель виконання, де інструкції виконуються послідовно: кожна операція блокує потік до свого завершення.

30. Що таке asynchronous programming?

Підхід, що дозволяє програмі виконувати інші задачі під час очікування завершення I/O-операцій без блокування основного потоку.

31. Для чого використовується async?

Ключове слово для оголошення асинхронної функції (корутини).

32. Для чого використовується await?

Ключове слово для прио зупинення виконання корутини до отримання результату від іншої асинхронної операції.

33. Що таке coroutine?

Спеціальний об'єкт функції в Python, виконання якої можна призупиняти та відновлювати.

34. Що таке event loop?

Цикл подій — ядро асинхронності в asyncio, яке керує розподілом часу та виконанням корутин і задач.

35. Що таке Task?

Обгортка над корутиною у asyncio, яка реєструє її в Event Loop для автономного паралельного виконання.

36. Для чого використовується asyncio.create_task()?

Для перетворення корутини на Task і негайного планування її виконання в event loop.

37. Для чого використовується asyncio.gather()?

Для одночасного (паралельного) запуску кількох асинхронних задач та очікування результатів усіх задач у вигляді списку.

38. Що таке timeout?

Граничний час, виділений на виконання операції, після закінчення якого операція переривається за виключенням.

39. Для чого потрібний retry?

Механізм автоматичного повторного відправлення запиту у разі виникнення тимчасової мережевої помилки.

40. Що таке backoff?

Стратегія затримки (наприклад, експоненціальна) між повторними спробами (retry) для уникнення перевантаження сервера.

41. Для чого використовується Semaphore?

Для обмеження кількості задач, які можуть одночасно отримувати доступ до певного ресурсу чи виконувати мережеві запити.

42. Що таке HTTPX?

Сучасний HTTP-клієнт для Python з повною підтримкою як синхронних, так і асинхронних (AsyncClient) запитів.

43. Чим httpx.AsyncClient відрізняється від синхронного клієнта?

Він не блокує потік під час мережевого обміну і працює з async/await.

44. Чому AsyncClient бажано повторно використовувати?

Щоб ефективно перевикористовувати підключення (Connection Pooling) та не витрачати ресурси на постійне відкриття TCP/TLS сесій.

45. Навіщо обмежувати паралелізм?

Щоб уникнути вичерпання системних ресурсів, сокетів, пам'яті або випадкового заблокування сторонніми сервісами через Rate Limiting (DOS).

46. Чому не слід запускати тисячі tasks одночасно?

Це призводить до великого споживання пам'яті та відмови мережевого стеку чи сервера БД через надмірне навантаження.

47. Як REST API взаємодіє з Repository Layer?

FastAPI ендпоінт створює/отримує DB Session через Dependency Injection, викликає відповідні методи репозиторію і повертає об'єкти до Pydantic для серіалізації.

48. Як FastAPI працює з Pydantic?

FastAPI автоматично конвертує вхідний JSON у Pydantic-моделі, здійснює валідацію, а також формує схему OpenAPI на їх основі.

49. Як протестувати REST API?

Використовуючи TestClient або httpx.AsyncClient з ASGITransport, роблячи запити до FastAPI застосунку в тестовому середовищі.

50. Чим API integration test відрізняється від unit test?

Unit test тестує окрему ізольовану функцію/метод; API integration test перевіряє взаємодію всього ланцюжка від HTTP-запиту, валідації Pydantic, логіки сервісів до запису у тестову БД.

10. Висновок

У лабораторній роботі №8 було успішно переведено проєкт обліку спортивних результатів на мережевий архітектурний рівень шляхом створення REST API на базі фреймворку **FastAPI**. Створено Pydantic-схеми для валідації вхідних даних, реалізовано повний набор CRUD endpoints (GET, POST, PATCH, DELETE), а також спеціальні ендпоінти для отримання рейтингів.

Асинхронну частину реалізовано за допомогою async/await, asyncio.Task, asyncio.gather та HTTP-клієнта **HTTPX**. Для захисту від мережевих збоїв та перевантажень додано механізми timeout, retry (з exponential backoff) та обмеження паралельних запитів за допомогою asyncio.Semaphore. Автоматизовані тести на pytest підтвердили працездатність та надійність створеного API.